CORE – 14: Algorithm Design Techniques (Unit – 1)

What is Algorithm? A finite set of instruction that specifies a sequence of operation is to be carried out in order to solve a specific problem or class of problems is called an Algorithm.

Characteristics of Algorithms

- **Input:** It should externally supply zero or more quantities.
- **Output:** It results in at least one quantity.
- **Definiteness:** Each instruction should be clear and unambiguous.
- **Finiteness:** An algorithm should terminate after executing a finite number of steps.
- **Flexibility:** It must be flexible enough to carry out desired changes with no efforts.
- **Efficient:** The term efficiency is measured in terms of time and space required by an algorithm to implement. Thus, an algorithm must ensure that it takes little time and less memory space for execution.
- **Independent:** An algorithm must be language independent, which means that it should mainly focus on the input and the procedure required to derive the output instead of depending upon the language.

Pseudocode: Pseudocode refers to an informal high-level description of the operating principle of a computer program or other algorithm. It uses structural conventions of a standard programming language intended for human reading rather than the machine reading.

Advantages of Pseudocode

- Since it is similar to a programming language, it can be quickly transformed into the actual programming language than a flowchart.
- The layman can easily understand it.
- Easily modifiable as compared to the flowcharts.
- Its implementation is beneficial for structured, designed elements.
- It can easily detect an error before transforming it into a code.

Disadvantages of Pseudocode

- Since it does not incorporate any standardized style or format, it can vary from one company to another.
- It does not depict the design.

Analysis of algorithm: The analysis is a process of estimating the efficiency of an algorithm. There are two fundamental parameters based on which we can analysis the algorithm:

- **Space Complexity:** The space complexity can be understood as the amount of space required by an algorithm to be executed.
- **Time Complexity:** Time complexity is a function of input size **n** that refers to the amount of time needed by an algorithm to be executed.

Generally, we make three types of analysis, which are as follows:

- **Worst-case time complexity:** For '**n**' input size, the worst-case time complexity can be defined as the maximum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the **maximum number of steps** performed on an instance having an input size of n.
- **Average case time complexity:** For '**n**' input size, the average-case time complexity can be defined as the average amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the **average number of steps** performed on an instance having an input size of n.
- **Best case time complexity:** For '**n**' input size, the best-case time complexity can be defined as the minimum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the minimum number of steps performed on an instance having an input size of n.

Asymptotic Notations: Asymptotic Notation is a way of comparing function that ignores constant factors and small input sizes. Three notations are used to calculate the running time complexity of an algorithm:

1. Big-oh Notation: Big-oh is the formal method of expressing the upper bound of an algorithm's running time. Mathematically it is defined as:

f(n) = O(g(n)) **[read as "f of n is big-oh of g of n"],** ***if there exists a positive constants c and n_0, such that $f(n) \leq c.g(n)$ $\forall$ $n \geq n_0$***

Example: $f(n) = 2n^2+5$ is $O(n^2)$

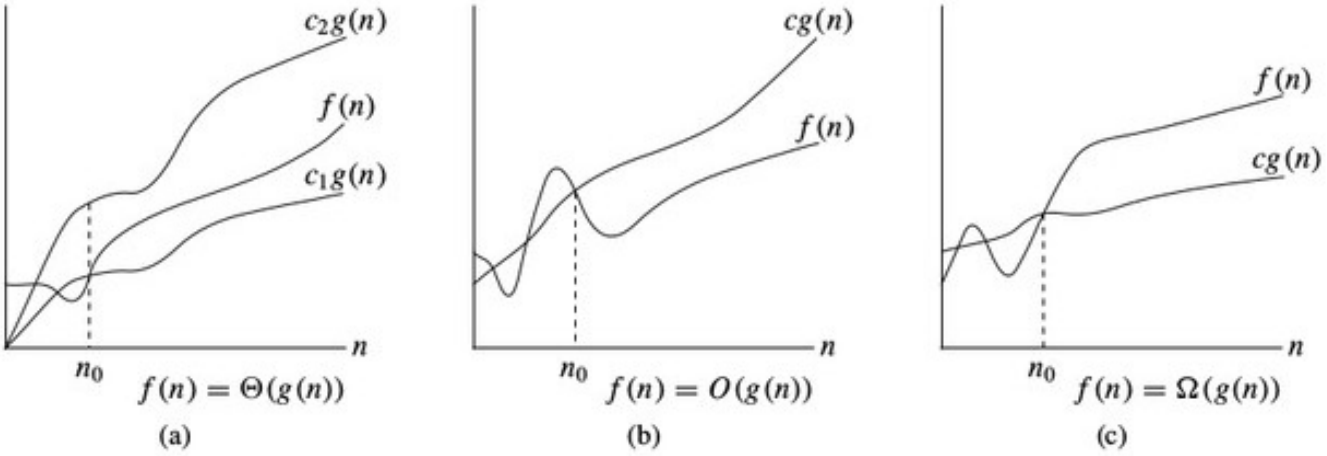
2. Big-Omega Notation: Big-oh is the formal method of expressing the lower bound of an algorithm's running time. Mathematically it is defined as:

f(n) = Ω(g(n)) **[read as "f of n is big-omega of g of n"],** ***if there exists a positive constants c and n_0, such that $0 \leq c.g(n) \leq f(n)$ $\forall$ $n \geq n_0$***

3. Theta (θ): The theta notation is both an upper and lower bound, so it defines exact asymptotic behaviour. Mathematically it is defined as:

f(n) = Θ(g(n)) **[read as "f of n is theta of g of n"],** ***if there exists a positive constants c_1, c_2 and n_0, such that $0 \leq c_1.g(n) \leq f(n) \leq c_2.g(n)$ $\forall$ $n \geq n_0$***

The above three asymptotic notations are pictorially represented below.

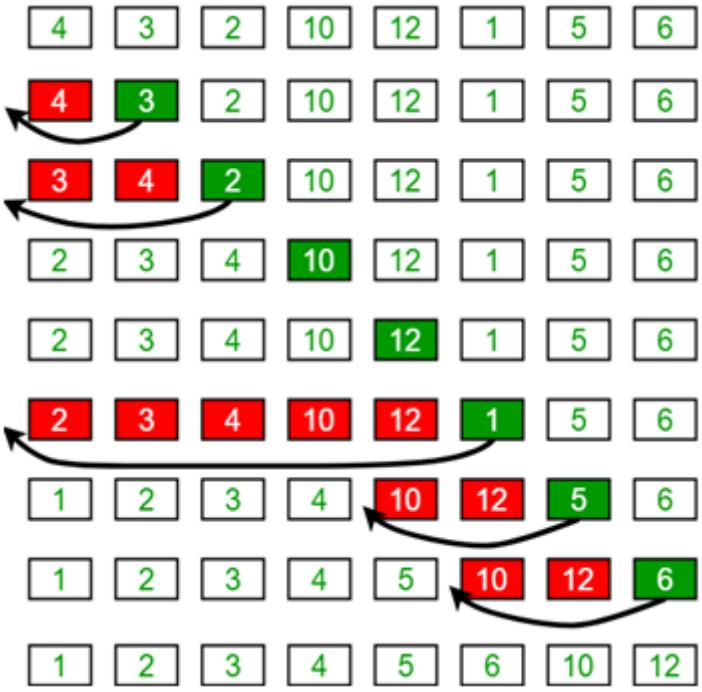


Analysis and design of Insertion sort algorithm:

ALGORITHM: INSERTION SORT (A)

1. For k ← 1 to length [A]
2. Do key ← A[k]
3. i = k-1
4. while (i ≥ 0 and A[i] > key) Repeat steps 5 and 6
    5. do A[i+1] ← A[i]
    6. i = i-1
7. A[i+1] ← key

Insertion sort execution **example**



Analysis:

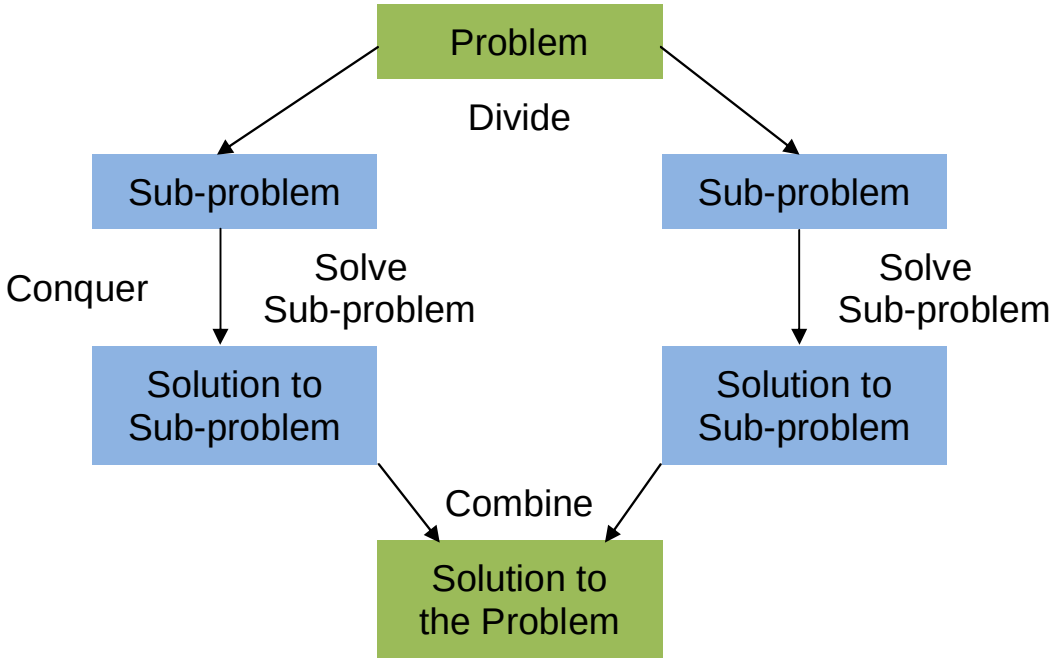
1. **Input:** n elements are given.
2. **Output:** the number of comparisons required to make sorting.
3. **Logic:** If we have n elements in insertion sort, then n-1 passes are required to find a sorted array.

In pass 1: no comparison is required
In pass 2: 1 comparison is required
In pass 3: 2 comparisons are required
.....
.....
In pass n: n-1 comparisons are required
Total comparisons: T (n) = 1+2+3+.....+ n-1
= (n-1)n / 2
= O (n^2)

Therefore complexity is of order n^2

Divide and Conquer paradigm: Divide and Conquer algorithm consists of a dispute using the following three steps.

1. **Divide** the original problem into a set of sub-problems of less size.
2. **Conquer:** Solve every sub-problem individually, recursively.
3. **Combine:** Put together the solutions of the sub-problems to get the solution to the whole problem.



Examples: Some of the algorithms based on the Divide & Conquer approach include:

1. Binary Search
2. Sorting (merge sort, quick sort)

The principles of Divide & Conquer Strategy are:

1. Relational Formula
2. Stopping Condition

Advantages of Divide and Conquer

- It minimizes the effort of designing an algorithm as it works on dividing the main problem into two or more sub-problems of less size and then solve them recursively.
- It efficiently uses cache memory without occupying much space because it solves simple sub-problems within the cache memory instead of accessing the slower main memory.

Disadvantages of Divide and Conquer

- An explicit stack may overuse the space.
- It may even crash the system if the recursion is performed rigorously greater than the stack present in the CPU.

Recurrence Relation: A recurrence is an equation or inequality that describes a function in terms of its values on smaller inputs. To solve a Recurrence Relation means to obtain a function defined on the natural numbers that satisfy the recurrence.

For example in Merge Sort, to sort a given array, we divide it in two halves and recursively repeat the process for the two halves. Finally we merge the results. Time complexity of Merge Sort can be written as T(n) = 2T(n/2) + cn. There are mainly three methods for solving Recurrences:

1. Substitution Method
2. Recursion Tree Method
3. Master Method

1. Substitution Method: The Substitution Method Consists of two main steps:

1. Guess the Solution.
2. Use the mathematical induction to find the boundary condition and shows that the guess is correct.

For example consider the recurrence T(n) = 2T(n/2) + n, for n>1

We guess the solution as T(n) = O(nLogn). Now we use induction to prove our guess.

We need to prove that T(n) ≤ cnLogn for some positive constant c.

We can assume that it is true for smaller values of n.

T(n) = 2T(n/2) + n

≤ 2cn/2Log(n/2) + n

= cn (Logn - Log2) + n

= cnLogn - cnLog2 + n

= cnLogn - cn + n

≤ cnLogn

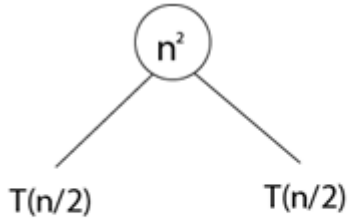
Thus **T (n) = O (n logn)** for n>1

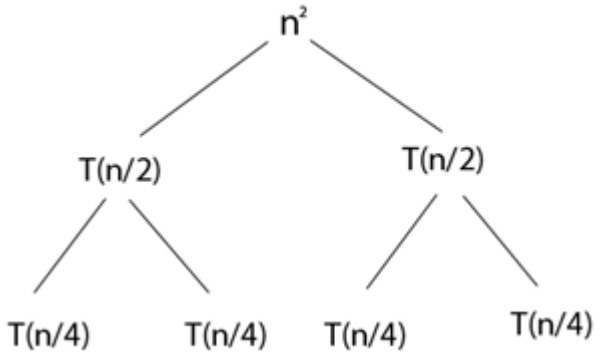
Recursion Tree Method: In this method, we draw a recurrence tree and calculate the time taken by every level of tree. Finally, we sum the work done at all levels. To draw the recurrence tree, we start from the given recurrence and keep drawing till we find a pattern among levels. The pattern is typically a arithmetic or geometric series. For example consider the recurrence relation

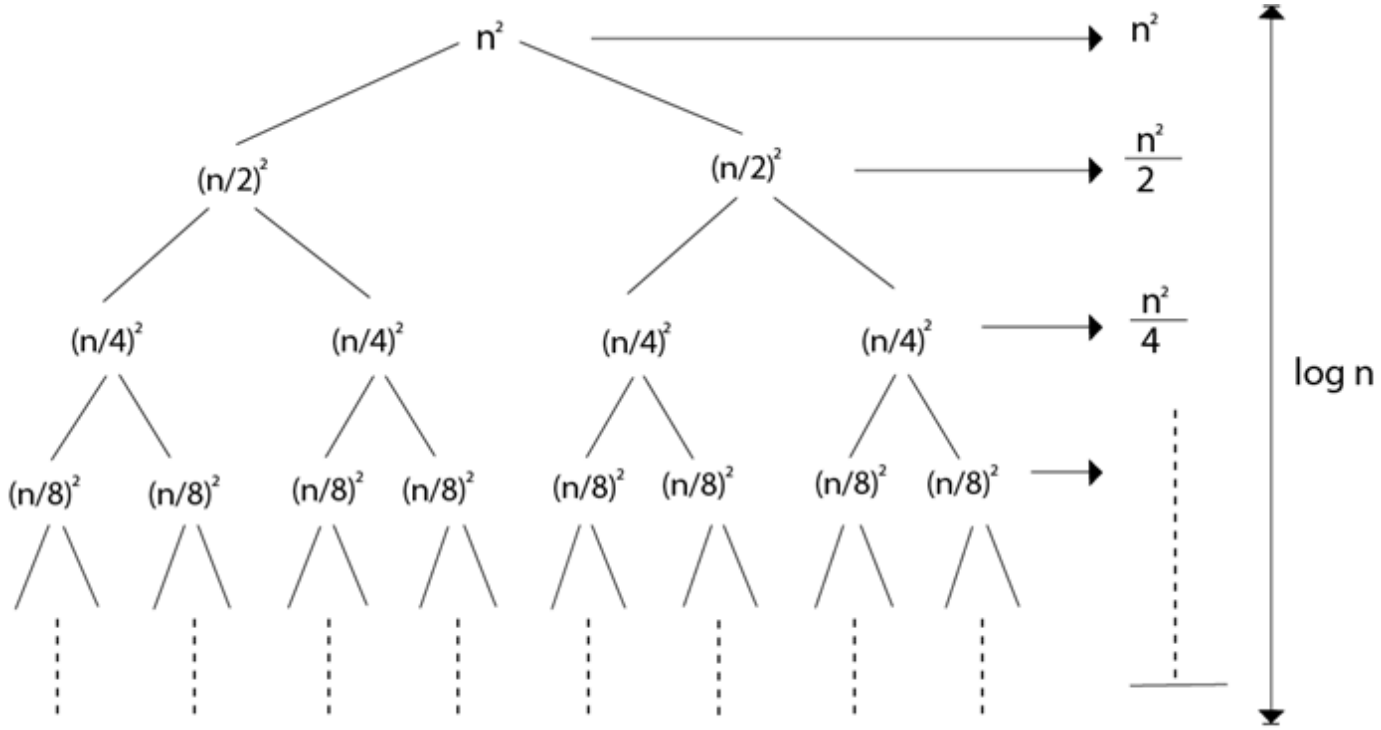
$$T(n) = 2T(n/2) + n^2$$

We have to obtain the asymptotic bound using recursion tree method.

Solution: The Recursion tree for the above recurrence is







$$T(n) = n^2 + \frac{n^2}{2} + \frac{n^2}{4} + \ldots\ldots \log n \text{ times.}$$

$$\leq n^2 \sum_{i=0}^{\infty} \left(\frac{1}{2^i}\right)$$

$$\leq n^2 \left(\frac{1}{1-\frac{1}{2}}\right) \leq 2n^2$$

$$\mathbf{T(n) = \theta n^2}$$

Master Method: The Master Method is used for solving the following types of recurrence

T(n) = aT(n/b) + f(n) for a≥1 and b>1 be non-negative integers & f(n) be a function.

Here, the significance of the literals are:

- n is the size of the problem.
- a is the number of sub-problems in the recursion.
- n/b is the size of each subproblem. (Here it is assumed that all sub-problems are essentially the same size.)
- f (n) is the sum of the work done outside the recursive calls, which includes the sum of dividing the problem and the sum of combining the solutions to the sub-problems.
- It is not possible always bound the function according to the requirement, so we make three cases which will tell us what kind of bound we can apply on the function.

Master Theorem: It is possible to complete an asymptotic tight bound in these three cases:

$$T(n)=\begin{cases}\Theta\left(n^{\log_b a}\right), & f(n)=O\left(n^{\log_b a-\varepsilon}\right)\\ \Theta\left(n^{\log_b a}\log n\right), & f(n)=\Theta\left(n^{\log_b a}\right)\\ \Theta\left(f(n)\right), & f(n)=\Omega\left(n^{\log_b a+\varepsilon}\right) \text{ and } af(n/b)\le cf(n) \text{ for large n}\end{cases}\quad \begin{matrix}\varepsilon>0\\ c<1\end{matrix}$$

Example-1: T (n) = 8T(n/2) + 1000n^2. Solve by applying master theorem.

Solution: Compare T (n) = 8T (n/2) + 1000n^2 with T (n) = aT (n/b) + f (n) for a≥1 & b>1
Here, a = 8, b=2, f (n) = 1000 n^2, $\log_b a = \log_2 8 = 3$

Put all the values in: f (n) = $O\left(n^{\log_b a-\varepsilon}\right)$

1000 n^2 = O ($n^{3-\varepsilon}$)
If we choose ε=1, we get: 1000 n^2 = O (n^{3-1}) = O (n^2)

Since this equation holds, the first case of the master theorem applies to the given recurrence relation, thus resulting in the conclusion:

T (n) = Θ $\left(n^{\log_b a}\right)$

Therefore: T (n) = Θ (n^3)

Example-2: T (n) = 2T (n/2) + n, Solve the recurrence by using the master method.
Compare T (n) = 2T (n/2) + n with T (n) = aT(n/b) + f(n) for a ≥ 1 b >1
Here, a = 2, b=2, f(n) = n, $\log_b a = \log_2 2 = 1$

We see that n = Θ $\left(n^{\log_b a}\right)$ = Θ (n) which is true. So case 2 of master theorem holds. Hence, T **(n) = Θ (n log n)**

Example-3: Solve the recurrence relation T(n) = 2T(n/2) + n^2 by using the master method.

Solution: Compare T (n) = 2T(n/2) + n^2 with T(n) = aT(n/b) + f(n) for a ≥ 1, b >1
Here, a = 2, b=2, f (n) = n^2, $\log_b a = \log_2 2 = 1$

Put all the values in f (n) = Ω $\left(n^{\log_b a+\varepsilon}\right)$..... (Eq. 1)

If we insert all the value in (Eq.1), we will get
n^2 = Ω($n^{1+\varepsilon}$), put ε =1, then the equality will hold.
n^2 = Ω(n^{1+1}) = Ω(n^2)
Now we will also check the second condition:

$2(n/2)^2 \le cn^2$

⇨ $n^2/2 \le cn^2$

If we will choose c =1/2, it is true that $n^2/2 \le n^2/2$ ∀ n ≥1
So it follows that T (n) = Θ ((f (n))
Hence, T (n) = Θ (n^2)